AI Gesture Mouse Project Report

This report documents the complete development process of the AI Gesture Mouse project. The project uses Computer Vision, MediaPipe hand tracking, OpenCV, voice recognition, and desktop GUI technologies to control the computer using hand gestures and voice commands.

---

1. Project Objective

```
The main objective of this project is to create a virtual AI-powered mouse system
     that allows users to control their computer using hand gestures instead of a physical mouse.
     The system also supports voice commands and desktop application integration.
```

2. Technologies Used

```
Python was used as the core programming language.
     OpenCV was used for webcam access and image processing.
     MediaPipe was used for hand landmark detection.
     PyAutoGUI was used for controlling mouse actions.
     SpeechRecognition and pyttsx3 were used for voice commands.
     CustomTkinter was used for desktop GUI development.
```

3. Project Folder Structure

```
GestureMouse/
       ■■■ src/
       ■   ■■■ app.py
       ■   ■■■ gesture.py
       ■   ■■■ hand_landmarker.task
       ■
       ■■■ assets/
       ■   ■■■ icon.ico
       ■   ■■■ logo.png
       ■
       ■■■ requirements.txt
       ■■■ README.md
```

4. Package Installation

```
Required installation commands:

     pip install opencv-python
     pip install mediapipe
     pip install pyautogui
     pip install speechrecognition
     pip install pyttsx3
     pip install pyaudio
     pip install customtkinter
     pip install pillow
     pip install pyinstaller
```

5. Role of Each Package

```
opencv-python:
     Used for webcam capture and real-time video processing.

     mediapipe:
     Detects hand landmarks and finger positions.
     pyautogui:
```

Controls mouse movement, clicks, scrolling, and keyboard actions.

speechrecognition:
Converts voice input into text commands.

pyttsx3:
Provides voice feedback from the system.

customtkinter:
Creates a modern desktop GUI.

pillow:
Used for image loading inside GUI.

pyinstaller:
Converts the Python application into a standalone EXE file.

6. MediaPipe Model Setup

The hand_landmarker.task model file is required for MediaPipe hand tracking.
The file must be placed inside the src folder.

Example:
src/hand_landmarker.task

7. Core Features Implemented

1. Cursor movement using index finger
2. Left click gesture
3. Right click gesture
4. Double click gesture
5. Scroll up and down
6. Swipe left and right navigation
7. Voice commands
8. Pause and resume system
9. Open Chrome gesture
10. Desktop GUI controls

8. Gesture Controls

Thumb + Index Finger:
Left Click

Thumb + Middle Finger:
Right Click

Two Finger Vertical Movement:
Scroll

Hand Swipe Left:
Previous Page

Hand Swipe Right:
Next Page

Thumbs Up:
Open Chrome

Peace Sign:
Pause or Resume

Open Palm:
Cursor Stop

9. Voice Commands

Supported voice commands:

```
- pause system
- resume system
- open chrome

Voice commands are processed using the SpeechRecognition library.
```

10. Desktop Application

```
A CustomTkinter desktop application was created.
     Features:
     - Start Button
     - Stop Button
     - Exit Button
     - Status Display
     - Dark Theme Interface
```

11. Important Files

```
app.py:
     Desktop application launcher.

     gesture.py:
     Main gesture recognition and AI control logic.

     hand_landmarker.task:
     MediaPipe AI model file.

     requirements.txt:
     List of required packages.

     README.md:
     Project documentation.
```

12. Example Core Code Snippet

```
pyautogui.moveTo(curr_x, curr_y)

     if click_distance < 30:
         pyautogui.click()

     if wrist.x > 0.85:
         pyautogui.hotkey('alt', 'right')
```

13. EXE File Creation

```
The desktop application was converted into a standalone EXE file using PyInstaller.

     Command:
     pyinstaller --onefile --windowed src/app.py

     Output:
     dist/app.exe
```

14. GitHub Integration

```
The project was uploaded to GitHub for version control and project showcase.

     Common Git commands used:
     git init
     git add .
     git commit -m "Initial commit"
     git push origin main
```

15. Problems Faced and Solutions

```
Problem:
     Indentation errors in gesture.py

     Solution:
     Corrected Python indentation structure.

     Problem:
     hand_landmarker.task not found

     Solution:
     Correct model path setup using os.path.join().

     Problem:
     Webcam not closing

     Solution:
     Added proper cleanup using:
     cap.release()
     cv2.destroyAllWindows()
```

16. Future Improvements

```
1. AI-based custom gesture learning
     2. Multi-hand tracking
     3. Gaming mode
     4. GPU acceleration
     5. Real-time gesture training
     6. Smart AI assistant integration
     7. Cloud synchronization
```

17. Final Conclusion

```
The AI Gesture Mouse project successfully demonstrates the use of Computer Vision,
     Artificial Intelligence, and Human Computer Interaction technologies.
     The system provides smooth gesture-based mouse control with voice assistance,
     making it a strong portfolio-level AI project.
```